2 HAFTA

C# ile Nesneye Yönelik Programlama · Ders Notu

Nesneye Yönelik Programlamaya (OOP) Giriş

Nesne Felsefesini Anlama

Bu haftanın kazanımları

1. **Class** ve **object** arasındaki farkı açıklayarak gerçek dünya varlıklarını sınıflarla modelleyebilir.
2. **public, private, protected, internal** erişim belirleyicilerini doğru yerde kullanabilir.
3. **Method** ve **property** tanımlayabilir; parametre türleri arasında seçim yapabilir.
4. **Constructor** yazabilir, aşırı yükleyebilir ve **this()** ile zincirleyebilir.
5. **Namespace** yapısını kurarak projeyi anlamlı birimlere ayırabilir.

İçindekiler

1 **Neden Nesneye Yönelik Programlama?**
Prosedürel yaklaşımın sınırları · Gerçek dünyayı modellemek · Dört temel prensibe bakış

2 **Sınıf (Class) ve Nesne (Object) Kavramları**
Şablon ve örnek ilişkisi · new anahtar kelimesi · Field kavramı · Stack ve heap üzerinde nesneler

3 **Erişim Belirleyiciler**
public · private · protected · internal · protected internal · private protected · Varsayılan değerler

4 **Metotlar ve Özellikler (Properties)**
Metot anatomisi · ref, out, in, params · Opsiyonel parametreler · Overloading · Auto-property · init ve required

5 **Constructor ve Constructor Türleri**
Varsayılan ve parametreli · Overloading · this() zinciri · static constructor · Primary constructor · Object initializer

6 **Namespace Yapısı ve Kullanımı**
using direktifi · File-scoped namespace · global using · Implicit usings · Ad çakışması ve alias

★ **Haftanın Özeti, Sözlük, Alıştırmalar ve Ekstra Okuma**
Özet · Terimler sözlüğü · Alıştırmalar · Kaynak bağlantıları

1. Neden Nesneye Yönelik Programlama?

Bu bölümün anahtar kelimeleri

procedural programming (prosedürel programlama) · **object-oriented programming** (nesneye yönelik programlama) · **abstraction** (soyutlama) · **modularity** (modülerlik) · **maintainability** (sürdürülebilirlik) · **reusability** (yeniden kullanılabilirlik)

1.1 Prosedürel yaklaşımın sınırları

Geçen hafta yazdığımız programlarda veriler ayrı değişkenlerde, işlemler ayrı metotlarda duruyordu. Küçük programlarda bu sorun değildir. Ancak bir öğrenci bilgi sistemi düşünün: her öğrenci için ad, numara, bölüm ve not listesi tutmanız gerekir. Prosedürel yaklaşımla bu şöyle görünür:

Kod 1.1 — Prosedürel yaklaşım: veriler dağınık

```
string[] adlar     = new string[100];
int[]    numaralar = new int[100];
string[] bolumler  = new string[100];
double[] ortalamalar = new double[100];

// 15. ogrencinin bilgilerini yazdirmak icin dort ayri diziye gitmek gerekir
Console.WriteLine($"{adlar[15]} - {numaralar[15]} - {ortalamalar[15]}");
```

Bu tasarımın üç büyük problemi vardır:

1. **Veri bütünlüğü yok.** `adlar[15]` ile `numaralar[15]`'in aynı kişiye ait olduğunu hiçbir şey garanti etmez. Bir diziden eleman silerseniz tüm diziler bozulur.
2. **Kod dağınık.** "Öğrenci" diye bir kavram kodda hiçbir yerde yoktur; sadece programcının zihnindedir.
3. **Değişime kapalı.** Öğrenciye "e-posta" alanı eklemek, tüm diziyi ve tüm metot imzalarını değiştirmeyi gerektirir.

1.2 OOP'nin çözümü: veri ile davranışı birleştirmek

OOP (Object-Oriented Programming / Nesneye Yönelik Programlama), ilgili verileri ve o veriler üzerinde çalışan işlemleri **tek bir birim** içinde toplar. Bu birime **class (sınıf)** denir.

Kod 1.2 — Aynı problem OOP ile

```
// Tek bir "Ogrenci" kavrami; veri ve davranis bir arada
class Ogrenci
{
    public string Ad { get; set; }
    public int Numara { get; set; }
    public string Bolum { get; set; }
    public double Ortalama { get; set; }

    public bool GectiMi() => Ortalama >= 50;   // davranis da burada
}

// Kullanim
Ogrenci o = new Ogrenci { Ad = "Elif", Numara = 2201, Ortalama = 78.4 };
Console.WriteLine($"{o.Ad} ({o.Numara}) - Gecti mi: {o.GectiMi()}");
```

> **[i] Anahtar fikir**
>
> Prosedürel programlamada sorduğunuz soru **"Hangi adımları uygulayacağım?"**dır. OOP'de sorduğunuz soru **"Sistemde hangi varlıklar var ve her biri neyi bilir, ne yapabilir?"**dır.

1.3 Bu haftaya ve gelecek haftaya bakış

OOP dört temel prensip üzerine kuruludur. Bu hafta bu prensiplere zemin hazırlayan yapı taşlarını öğreneceğiz; prensiplerin kendisini 3. haftada detaylı işleyeceğiz.

Tablo 1.1 — OOP'nin dört temel prensibi (önizleme)

Prensip	Türkçesi	Tek cümlede
Encapsulation	Kapsülleme	Veriyi gizle, ona sadece kontrollü kanallardan eriştir.
Inheritance	Kalıtım	Var olan bir sınıfın özelliklerini devralarak yenisini türet.
Polymorphism	Çok biçimlilik	Aynı çağrı, nesnenin gerçek tipine göre farklı davransın.
Abstraction	Soyutlama	Gereksiz ayrıntıyı gizle, sadece gerekli sözleşmeyi göster.

2. Sınıf (Class) ve Nesne (Object) Kavramları

Bu bölümün anahtar kelimeleri

class (sınıf) · **object** (nesne) · **instance** (örnek) · **instantiation** (örnekleme) · **field** (alan) · **member** (üye) · **blueprint** (şablon / plan) · **reference** (referans) · **null reference** (boş referans)

2.1 Şablon ve örnek ilişkisi

Class (sınıf) bir **blueprint (şablon)**'tur: neyin nasıl olacağını tarif eder ama kendisi bellekte veri tutmaz. **Object (nesne)** ise bu şablondan üretilmiş somut bir **instance (örnek)**'tir.

Benzetme	Class	Object
Mimari	Ev projesi (çizim)	O projeye göre inşa edilmiş 3 ayrı ev
Mutfak	Kek tarifi	Tarife göre pişirilen her bir kek
Kimlik	Kimlik kartı formatı	Ali'nin kimliği, Ayşe'nin kimliği

Kod 2.1 — İlk sınıfımız

```
// SINIF TANIMI (sablon)
class Kitap
{
    // Field (alan): sinifin veri tutan uyeleri
    public string Baslik;
    public string Yazar;
    public int SayfaSayisi;

    // Method (metot): sinifin davranisi
    public void BilgiYazdir()
    {
        Console.WriteLine($"{Baslik} / {Yazar} ({SayfaSayisi} sayfa)");
    }
}

// NESNE OLUSTURMA (instantiation)
Kitap k1 = new Kitap();          // new -> heap'te yer ayirir
k1.Baslik = "Tutunamayanlar";
k1.Yazar = "Oguz Atay";
k1.SayfaSayisi = 724;

Kitap k2 = new Kitap();          // tamamen bagimsiz ikinci nesne
k2.Baslik = "Kuyucakli Yusuf";
k2.Yazar = "Sabahattin Ali";
k2.SayfaSayisi = 220;

k1.BilgiYazdir();
k2.BilgiYazdir();
```

Kod 2.1 çıktısı

```
Tutunamayanlar / Oguz Atay (724 sayfa)
Kuyucakli Yusuf / Sabahattin Ali (220 sayfa)
```

2.2 Nesneler bellekte nasıl durur?

1. haftada gördüğümüz **value type / reference type** ayrımı burada hayati önem kazanır. `class` ile tanımlanan her tip bir **reference type**'tır.

Kod 2.2 — Bellek şeması

```
Kitap k1 = new Kitap();
Kitap k2 = k1;              // ADRES kopyalanir, yeni nesne OLUSMAZ

        STACK                          HEAP
   +-------------+          +---------------------------+
   | k1  0x4A20  | -------> | Baslik: "Tutunamayanlar"  |
   +-------------+     +--> | Yazar : "Oguz Atay"       |
   | k2  0x4A20  | ----+    | SayfaSayisi: 724          |
   +-------------+          +---------------------------+

k2.SayfaSayisi = 999;
Console.WriteLine(k1.SayfaSayisi);   // 999 -> ayni nesne!
```

> **[x] En sık görülen çalışma zamanı hatası: NullReferenceException**
>
> Bir referans değişkeni tanımlamak nesne oluşturmaz. `Kitap k;` yazıp hemen `k.Baslik` demek `NullReferenceException` fırlatır. Nesne ancak `new` ile bellekte yer ayrıldığında var olur.

Kod 2.3 — Null kontrolü

```
Kitap? k = BulKitap("12345");      // bulamazsa null dondurdugunu varsayalim

if (k is not null)
    Console.WriteLine(k.Baslik);

Console.WriteLine(k?.Baslik ?? "Kitap bulunamadi");   // tek satirda guvenli
```

2.3 Field (alan) ve isimlendirme

Field, sınıfın doğrudan veri tutan üyesidir. Modern C#'ta field'lar genellikle `private` yapılır ve dışarıya **property** üzerinden açılır — bunun nedenini 3. haftada Encapsulation konusunda göreceğiz.

Kod 2.4

```
class Urun
{
    private string _ad;          // private field: _camelCase
    private decimal _fiyat;
    public const decimal KDVOrani = 0.20m;   // sabit: PascalCase
    public static int ToplamUrunSayisi;      // static: tum nesnelerde ORTAK
}
```

> **[i] static ne demek?**
>
> `static` üyeler **nesneye değil sınıfa** aittir. `ToplamUrunSayisi` için bellekte tek bir kopya vardır ve tüm `Urun` nesneleri onu paylaşır. Erişim `Urun.ToplamUrunSayisi` şeklinde, nesne üzerinden değil sınıf adı üzerinden yapılır. `Math.PI` ve `Console.WriteLine` da birer static üyedir.

3. Erişim Belirleyiciler (Access Modifiers)

Bu bölümün anahtar kelimeleri

`access modifier` (erişim belirleyici) · `visibility` (görünürlük) · `assembly` (derleme birimi) · `encapsulation` (kapsülleme) · `information hiding` (bilgi gizleme)

3.1 Neden erişimi kısıtlarız?

Bir sınıfın her üyesini `public` yapmak, o sınıfın **iç işleyişini** dış dünyaya açmak demektir. Bu durumda sınıfın içini değiştirdiğinizde onu kullanan tüm kodlar bozulur. Erişim belirleyiciler, sınıfın **dışarıya verdiği söz** ile **içeride nasıl çalıştığını** birbirinden ayırmanızı sağlar.

3.2 Altı erişim belirleyici

Tablo 3.1 — Erişim matrisi

Belirleyici	Aynı sınıf	Türetilmiş sınıf (aynı assembly)	Aynı assembly	Türetilmiş sınıf (başka assembly)	Her yer
`private`	✓	✗	✗	✗	✗
`private protected`	✓	✓	✗	✗	✗
`protected`	✓	✓	✗	✓	✗
`internal`	✓	✓	✓	✗	✗
`protected internal`	✓	✓	✓	✓	✗
`public`	✓	✓	✓	✓	✓

Assembly (derleme birimi), derleme sonucunda oluşan tek bir `.dll` veya `.exe` dosyasıdır. Pratikte "bir proje" demektir.

En çok kullandığınız üçü

- `private` — Yalnızca tanımlandığı sınıfın içinden erişilir. **Field'lar için varsayılan tercihiniz bu olmalı.**
- `public` — Her yerden erişilir. Sınıfın dış dünyaya sunduğu **API**'dir; dikkatli seçilmelidir.
- `protected` — Sınıfın kendisi ve ondan türeyen sınıflar erişebilir. Kalıtım (3. hafta konusu) için tasarlanmıştır.

3.3 Varsayılan değerler

Tablo 3.2 — Belirleyici yazmazsanız ne olur?

Öğe	Varsayılan erişim
Sınıf üyeleri (field, method, property)	`private`
Üst düzey (top-level) sınıf, interface, enum	`internal`
`interface` üyeleri	`public` (değiştirilemez, C# 8 öncesi)
`enum` üyeleri	`public`

Kod 3.1 — Erişim belirleyiciler iş başında

```
public class BankaHesabi
{
    private decimal _bakiye;              // disaridan GORUNMEZ
    public string HesapNo { get; private set; }  // disaridan okunur, yazilamaz

    public BankaHesabi(string hesapNo)
    {
        HesapNo = hesapNo;
    }

    public void ParaYatir(decimal tutar)   // kontrollu giris kapisi
    {
        if (tutar <= 0)
        {
            Console.WriteLine("Tutar pozitif olmalidir.");
            return;
        }
        _bakiye += tutar;
        Logla($"{tutar:C} yatirildi.");
    }

    public decimal BakiyeGoruntule() => _bakiye;

    private void Logla(string mesaj)       // ic detay, disariya kapali
        => Console.WriteLine($"[LOG] {DateTime.Now:HH:mm} {mesaj}");
}

// --- Kullanim ---
var hesap = new BankaHesabi("TR1234");
hesap.ParaYatir(500);
// hesap._bakiye = 1_000_000;   // DERLEME HATASI: _bakiye private
// hesap.Logla("deneme");       // DERLEME HATASI: Logla private
// hesap.HesapNo = "TR9999";    // DERLEME HATASI: set private
Console.WriteLine(hesap.BakiyeGoruntule());
```

> **[+] Altın kural**
>
> Bir üyeyi **mümkün olan en dar** erişimle başlatın. Sonradan `private`'ı `public` yapmak kolaydır; `public`'i `private` yapmak ise o üyeyi kullanan tüm kodları bozar.

4. Metotlar ve Özellikler (Properties)

Bu bölümün anahtar kelimeleri

method (metot) · **parameter** (parametre) · **argument** (argüman) · **return type** (dönüş tipi) · **signature** (imza) · **overloading** (aşırı yükleme) · **property** (özellik) · **getter / setter** (okuyucu / yazıcı) · **backing field** (arka plan alanı) · **expression-bodied** (ifade gövdeli)

4.1 Metot anatomisi

Kod 4.1 — Bir metodun parçaları

```
public   double   Hesapla   (int adet, decimal birimFiyat)
//  |       |       |              |
//  |       |       |              +-- parameter list (parametre listesi)
//  |       |       +----------------- method name (metot adi, PascalCase)
//  |       +------------------------- return type (donus tipi)
//  +--------------------------------- access modifier (erisim belirleyici)
{
    return (double)(adet * birimFiyat);   // return: degeri geri dondurur
}

// void: geri deger dondurmeyen metot
public void Selamla(string ad)
{
    Console.WriteLine($"Merhaba {ad}");
}
```

> **[i] Signature (imza) nedir?**
>
> Bir metodun imzası **adı + parametre tipleri ve sırası**ndan oluşur. **Dönüş tipi imzaya dâhil değildir.** Bu yüzden sadece dönüş tipi farklı iki metot tanımlayamazsınız.

4.2 Parametre türleri

Tablo 4.1 — Parametre geçirme yöntemleri

Anahtar kelime	Anlamı	Çağırırken atanmış olmalı mı?
(yok)	Değer kopyalanır (value type) veya referans kopyalanır (reference type)	Evet
`ref`	Değişkenin kendisi geçirilir; metot onu değiştirebilir	Evet
`out`	Metot değeri **doldurmak zorundadır**	Hayır
`in`	Referansla geçirilir ama **değiştirilemez** (performans için)	Evet
`params`	Değişken sayıda argüman alır; son parametre olmalı	—

Kod 4.2

```
// ref: degiskeni metot icinde degistirebilir
static void IkiyeKatla(ref int sayi) => sayi *= 2;
```

```
// out: metot degeri doldurmak ZORUNDA
static bool BolmeYap(int a, int b, out int sonuc)
{
    if (b == 0) { sonuc = 0; return false; }
    sonuc = a / b;
    return true;
}

// params: kac tane argüman gelecegi belli degil
static int Topla(params int[] sayilar)
{
    int toplam = 0;
    foreach (int s in sayilar) toplam += s;
    return toplam;
}

// --- Kullanim ---
int x = 5;
IkiyeKatla(ref x);
Console.WriteLine(x);                    // 10

if (BolmeYap(10, 2, out int bolum))
    Console.WriteLine(bolum);            // 5

Console.WriteLine(Topla(1, 2, 3, 4, 5));  // 15
Console.WriteLine(Topla());               // 0
```

Opsiyonel ve adlandırılmış parametreler

Kod 4.3

```
static void Rapor(string baslik, bool detayli = false, string format = "txt")
{
    Console.WriteLine($"{baslik} | detay={detayli} | format={format}");
}

Rapor("Satis");                              // Satis | detay=False | format=txt
Rapor("Satis", true);                        // Satis | detay=True  | format=txt
Rapor("Satis", format: "pdf");               // named argument (adlandirilmis)
Rapor(baslik: "Satis", detayli: true, format: "csv");
```

4.3 Method overloading (aşırı yükleme)

Aynı isimli birden fazla metodu **farklı parametre imzalarıyla** tanımlayabilirsiniz. Derleyici, çağrıdaki argümanlara bakarak hangisini çalıştıracağına **derleme zamanında** karar verir. Bu, 3. haftada göreceğimiz **compile-time polymorphism**'in ta kendisidir.

Kod 4.4

```
class Hesaplayici
{
    public int Topla(int a, int b) => a + b;
    public double Topla(double a, double b) => a + b;
    public int Topla(int a, int b, int c) => a + b + c;
    public string Topla(string a, string b) => a + b;

    // public double Topla(int a, int b) { ... }  // HATA!
    // Sadece donus tipi farkli olamaz - imza ayni.
}

var h = new Hesaplayici();
Console.WriteLine(h.Topla(3, 4));          // 7    -> int versiyonu
Console.WriteLine(h.Topla(3.5, 4.2));      // 7.7  -> double versiyonu
Console.WriteLine(h.Topla(1, 2, 3));       // 6    -> uc parametreli
Console.WriteLine(h.Topla("C", "#"));      // C#   -> string versiyonu
```

4.4 Properties (özellikler)

Property, dışarıdan bakıldığında bir field gibi kullanılan, ancak arka planda metot gibi çalışan üyedir. `get` bloğu okuma, `set` bloğu yazma işlemini yönetir. Bu sayede **araya kontrol kodu yerleştirebilirsiniz**.

Kod 4.5 — Property'nin üç yazım biçimi

```
class Personel
{
    // 1) FULL PROPERTY - backing field ile, tam kontrol
    private decimal _maas;
    public decimal Maas
    {
        get { return _maas; }
        set
        {
            if (value < 0)
                throw new ArgumentException("Maas negatif olamaz.");
            _maas = value;     // 'value' = atanan deger (ortuk parametre)
        }
    }

    // 2) AUTO-IMPLEMENTED PROPERTY - derleyici field'i kendi olusturur
    public string Ad { get; set; } = "";

    // 3) HESAPLANAN (read-only) PROPERTY - sadece get
    public decimal YillikMaas => Maas * 12;
}

var p = new Personel { Ad = "Deniz", Maas = 45_000m };
Console.WriteLine(p.YillikMaas);      // 540000
// p.Maas = -100;                     // ArgumentException firlatir
```

Erişimi kısıtlama, init ve required

Kod 4.6

```
class Siparis
{
    public int Id { get; private set; }     // disaridan okunur, sadece icerden yazilir
    public DateTime Tarih { get; init; }    // SADECE olusturulurken atanabilir
    public required string MusteriAdi { get; set; }  // atanmasi ZORUNLU

    public Siparis(int id)
    {
        Id = id;
    }
}

var s = new Siparis(1)
{
    MusteriAdi = "Kutlu A.S.",   // required: yazilmazsa derleme hatasi
    Tarih = DateTime.Now         // init: sadece burada atanabilir
};
// s.Tarih = DateTime.Today;     // HATA: init-only property
```

> **[+] Field mı, property mi?**
>
> Kural basittir: **public field yazmayın.** Dışarıya açılacak her veri bir property olmalıdır. Bugün basit bir `{ get; set; }` olsa bile, yarın araya doğrulama eklemeniz gerektiğinde sınıfı kullanan kodları değiştirmek zorunda kalmazsınız.

5. Constructor ve Constructor Türleri

Bu bölümün anahtar kelimeleri

constructor (yapıcı metot) · **default constructor** (varsayılan yapıcı) · **parameterized constructor** (parametreli yapıcı) · **constructor chaining** (yapıcı zincirleme) · **initialization** (ilk değer atama) · **object initializer** (nesne başlatıcı) · **primary constructor** (birincil yapıcı)

5.1 Constructor nedir?

Constructor (yapıcı metot), `new` ile nesne oluşturulduğu anda otomatik çalışan özel bir metottur. Görevi, nesnenin **geçerli bir başlangıç durumunda** doğmasını sağlamaktır.

- Adı **sınıf adıyla birebir aynı** olmak zorundadır.
- **Dönüş tipi yazılmaz** — `void` bile değil.
- Aşırı yüklenebilir (overload edilebilir).
- Siz hiç yazmazsanız, derleyici **parametresiz varsayılan bir constructor** üretir.

> **[!] Parametreli constructor yazarsanız varsayılanı kaybedersiniz**
>
> En az bir constructor tanımladığınız anda derleyici artık varsayılan (parametresiz) constructor'ı üretmez. `new Ogrenci()` yazmaya devam etmek istiyorsanız onu da elle eklemelisiniz.

5.2 Constructor türleri

Kod 5.1 — Beş farklı constructor bir arada

```
class Ogrenci
{
    public string Ad { get; set; }
    public int Numara { get; set; }
    public string Bolum { get; set; }
    public static int ToplamOgrenci { get; private set; }

    // 1) STATIC CONSTRUCTOR
    // Sinif ilk kez kullanildiginda BIR KEZ calisir. Parametre alamaz,
    // erisim belirleyici yazilamaz.
    static Ogrenci()
    {
        ToplamOgrenci = 0;
        Console.WriteLine("[static ctor] Ogrenci sinifi hazirlandi.");
    }

    // 2) DEFAULT (parametresiz) CONSTRUCTOR
    public Ogrenci()
    {
        Ad = "Isimsiz";
        Bolum = "Belirtilmedi";
        ToplamOgrenci++;
    }

    // 3) PARAMETRELI CONSTRUCTOR
    public Ogrenci(string ad, int numara, string bolum)
```

```
    {
        Ad = ad;
        Numara = numara;
        Bolum = bolum;
        ToplamOgrenci++;
    }

    // 4) CONSTRUCTOR CHAINING - this() ile digerini cagirir
    // Kod tekrarini onler: asil is 3 numarali ctor'da yapilir.
    public Ogrenci(string ad, int numara)
        : this(ad, numara, "Bilgisayar Muhendisligi")
    {
    }

    // 5) COPY CONSTRUCTOR - var olan nesneden yenisini uretir
    public Ogrenci(Ogrenci baska)
        : this(baska.Ad, baska.Numara, baska.Bolum)
    {
    }
}
```

Kod 5.2 — Kullanımı ve çıktısı

```
var o1 = new Ogrenci();
var o2 = new Ogrenci("Elif", 2201, "Endustri Muhendisligi");
var o3 = new Ogrenci("Mert", 2202);          // bolum varsayilan atanir
var o4 = new Ogrenci(o2);                    // kopya

Console.WriteLine(o3.Bolum);
Console.WriteLine($"{o4.Ad} / {o4.Bolum}");
Console.WriteLine($"Toplam ogrenci: {Ogrenci.ToplamOgrenci}");
```

Kod 5.2 çıktısı

```
[static ctor] Ogrenci sinifi hazirlandi.
Bilgisayar Muhendisligi
Elif / Endustri Muhendisligi
Toplam ogrenci: 4
```

5.3 Primary constructor (C# 12+)

Sınıf adının hemen yanına yazılan parametreler, sınıfın tamamında kullanılabilir. Kısa sınıflarda ciddi kod tasarrufu sağlar.

Kod 5.3

```
// Klasik yazim
class Nokta
{
    public int X { get; }
    public int Y { get; }
    public Nokta(int x, int y) { X = x; Y = y; }
}

// Primary constructor ile ayni sey
class Nokta2(int x, int y)
{
    public int X { get; } = x;
    public int Y { get; } = y;
    public double Uzaklik() => Math.Sqrt(x * x + y * y);   // x, y dogrudan kullanilir
}

var n = new Nokta2(3, 4);
Console.WriteLine(n.Uzaklik());     // 5
```

5.4 Object initializer (nesne başlatıcı)

Constructor yazmadan, nesne oluşturulduktan hemen sonra property'lere toplu değer atamanın kısa yoludur. Süslü parantez bloğu, constructor çalıştıktan **sonra** işletilir.

Kod 5.4

```
// Object initializer: her property icin ayri satir yazmaya gerek yok
var o = new Ogrenci
{
    Ad = "Yagmur",
    Numara = 2210,
    Bolum = "Yazilim Muhendisligi"
};

// Koleksiyon initializer ile birlikte
List<Ogrenci> sinif =
[
    new Ogrenci { Ad = "Ali",  Numara = 1 },
    new Ogrenci { Ad = "Ayse", Numara = 2 }
];
```

> **[i] Constructor mı, object initializer mı?**
>
> Nesnenin **geçerli olması için zorunlu** olan veriler constructor parametresi olmalıdır (örneğin hesap numarası olmayan bir banka hesabı anlamsızdır). **İsteğe bağlı** alanlar object initializer ile atanabilir. `required` anahtar kelimesi bu ikisinin arasını birleştirir.

6. Namespace Yapısı ve Kullanımı

Bu bölümün anahtar kelimeleri

namespace (ad alanı) · **using directive** (using yönergesi) · **fully qualified name** (tam nitelikli ad) · **alias** (takma ad) · **name collision** (ad çakışması) · **global using** (genel using)

6.1 Namespace neden var?

Büyük bir projede yüzlerce sınıf olur ve isimler kaçınılmaz olarak çakışır. **Namespace (ad alanı)**, sınıfları mantıksal gruplara ayırarak bu çakışmayı önler. Tıpkı bilgisayardaki klasörler gibi düşünebilirsiniz: aynı isimli iki dosya farklı klasörlerde sorunsuz durabilir.

Kod 6.1 — İki yazım biçimi

```
// KLASIK (block-scoped) namespace
namespace Okul.Yonetim
{
    public class Ogrenci { }
}

// FILE-SCOPED namespace (C# 10+) - bir girinti seviyesi kazandirir
// Dosyada sadece BIR namespace olabilir ve en ustte yazilir.
namespace Okul.Yonetim;

public class Ogrenci { }
public class Ogretmen { }
```

6.2 using direktifi

Kod 6.2

```
using System;                          // temel using
using System.Collections.Generic;
using Okul.Yonetim;                    // kendi namespace'imiz

using static System.Math;              // static using: sinif adi yazmadan kullan
using Ogr = Okul.Yonetim.Ogrenci;      // ALIAS (takma ad)

class Program
{
    static void Main()
    {
        Console.WriteLine(Sqrt(16));   // Math.Sqrt yerine sadece Sqrt
        Ogr o = new Ogr();             // Okul.Yonetim.Ogrenci yerine Ogr

        // using yazmadan kullanmak isterseniz: FULLY QUALIFIED NAME
        Okul.Yonetim.Ogretmen t = new Okul.Yonetim.Ogretmen();
    }
}
```

6.3 global using ve implicit usings

`global using` ile bir `using` satırını projedeki **tüm dosyalar** için geçerli kılabilirsiniz. .NET 6 ve sonrasında oluşturulan projelerde **implicit usings (örtük using'ler)** varsayılan olarak açıktır; bu yüzden `using System;` yazmadan da `Console` kullanabiliyorsunuz.

Kod 6.3

```
// GlobalUsings.cs adinda tek bir dosyada toplamak yaygin bir pratiktir
global using System;
global using System.Collections.Generic;
global using Okul.Yonetim;
```

Kod 6.4 — .csproj dosyasındaki ilgili ayar

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

6.4 Klasör yapısı ile namespace ilişkisi

Zorunlu olmamakla birlikte, **klasör adları ile namespace'lerin birebir eşleşmesi** güçlü bir gelenektir. Visual Studio yeni sınıf eklerken namespace'i klasör yoluna göre otomatik üretir.

Kod 6.5 — Önerilen proje düzeni

```
Okul.Yonetim/                    -> namespace Okul.Yonetim
+-- Okul.Yonetim.csproj
+-- Models/                      -> namespace Okul.Yonetim.Models
|   +-- Ogrenci.cs
|   +-- Ogretmen.cs
+-- Services/                    -> namespace Okul.Yonetim.Services
|   +-- NotHesaplayici.cs
+-- Program.cs
```

> **[+] İsimlendirme geleneği**
>
> Namespace'ler genellikle **Şirket.Ürün.Modül** düzeninde adlandırılır: `Microsoft.Extensions.Logging`, `Kutlu.Saha.Models` gibi. Tümü **PascalCase** yazılır ve çoğul/tekil tutarlılığına dikkat edilir.

Haftanın Özeti

1. **Class** bir şablon, **object** o şablondan üretilen somut örnektir. Nesne ancak `new` ile bellekte var olur.
2. Sınıflar **reference type**'tır: iki değişken aynı nesneyi gösterebilir ve atanmamış bir referans `NullReferenceException`'a yol açar.
3. **Erişim belirleyiciler** sınıfın iç işleyişini dış dünyadan ayırır. Field'lar `private`, dışa açılan üyeler `public` olmalıdır.
4. **Metotlar** davranışı, **property'ler** kontrollü veri erişimini sağlar. `ref`, `out`, `in`, `params` parametre davranışını değiştirir.
5. **Overloading**, aynı isimli metotları farklı imzalarla tanımlamaktır; karar derleme zamanında verilir.
6. **Constructor** nesnenin geçerli bir durumda doğmasını garanti eder. `this()` ile zincirlenebilir, `static` olanı sınıf başına bir kez çalışır.
7. **Namespace**'ler ad çakışmasını önler ve projeyi mantıksal birimlere ayırır. Modern C#'ta **file-scoped namespace** ve **global using** tercih edilir.

Anahtar Terimler Sözlüğü

Terim (EN)	Türkçe	Kısa tanım
class	sınıf	Nesnelerin şablonu; veri ve davranışı bir arada tanımlar
object / instance	nesne / örnek	Sınıftan `new` ile üretilen somut varlık
field	alan	Sınıfın doğrudan veri tutan üyesi
property	özellik	get/set ile kontrollü erişim sağlayan üye
backing field	arka plan alanı	Bir property'nin değerini saklayan private field
method	metot	Sınıfın davranışını tanımlayan kod bloğu
signature	imza	Metot adı + parametre tipleri; dönüş tipi dâhil değildir
overloading	aşırı yükleme	Aynı adlı metodun farklı imzalarla tanımlanması
constructor	yapıcı metot	Nesne oluşturulurken otomatik çalışan metot
access modifier	erişim belirleyici	Üyenin nereden görülebileceğini belirler
assembly	derleme birimi	Derleme sonucu oluşan .dll veya .exe
namespace	ad alanı	Sınıfları mantıksal gruplara ayıran yapı
static	statik	Nesneye değil sınıfa ait olan üye

Alıştırmalar

1. **Dikdortgen** adında bir sınıf yazın. `Genislik` ve `Yukseklik` property'leri, `AlanHesapla()` ve `CevreHesapla()` metotları olsun. Negatif değer atanmasını engelleyin.
2. **Kitap** sınıfına üç farklı constructor ekleyin: parametresiz, sadece başlık alan, başlık + yazar + sayfa alan. Kod tekrarını `this()` zincirlemesiyle önleyin.
3. **Daire** sınıfında `Yaricap` property'si olsun. `Alan` ve `Cevre` hesaplanan (read-only) property olarak yazılsın.
4. **Sayac** adında bir sınıf yazın. Kaç adet `Sayac` nesnesi oluşturulduğunu `static` bir sayaçla takip etsin.
5. **Hesaplayici** sınıfına `Carp` metodunu dört kez aşırı yükleyin: `(int,int)`, `(double,double)`, `(int,int,int)` ve `params int[]`.

6. Bir **Urun** sınıfı yazın: `Ad` ve `Fiyat` property'leri, `KdvliFiyat` hesaplanan property'si olsun. Ürünleri bir `List<Urun>` içinde tutup toplam tutarı yazdırın.
7. **Araştırma:** `required` ile constructor parametresi arasındaki fark nedir? Hangi durumda hangisini seçersiniz?

Ekstra Okuma ve Kaynaklar

- **Microsoft Learn — Nesne Yönelimli Programlamaya Giriş (Türkçe)**
  Sınıf, nesne ve kalıtımı adım adım bir banka hesabı örneği üzerinden anlatan resmî eğitim.
  https://learn.microsoft.com/tr-tr/dotnet/csharp/fundamentals/tutorials/oop
- **Classes and objects — C# Belgeleri**
  Sınıf tanımı, referans semantiği ve nesne yaşam döngüsünün resmî açıklaması.
  https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/classes
- **Access Modifiers — C# Programming Guide**
  Altı erişim belirleyicinin tamamı, hangi bağlamda hangisinin kullanılabileceği ile birlikte.
  https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/access-modifiers
- **Properties — C# Programming Guide**
  Auto-property, init-only, required ve hesaplanan property'lerin ayrıntılı anlatımı.
  https://learn.microsoft.com/en-us/dotnet/csharp/properties
- **Constructors — C# Programming Guide**
  Instance, static ve primary constructor'lar; çalışma sıraları ve zincirleme.
  https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constructors
- **Primary constructors — C# 12 yeniliği**
  Primary constructor'ın ne zaman uygun olduğunu örneklerle karşılaştıran rehber.
  https://learn.microsoft.com/en-us/dotnet/csharp/whats-new/tutorials/primary-constructors
- **Namespaces — C# Programming Guide**
  Namespace tanımı, iç içe namespace'ler ve alias kullanımı.
  https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/namespaces
- **C# Coding Conventions — İsimlendirme kuralları**
  PascalCase, camelCase ve _camelCase'in nerede kullanılacağına dair resmî rehber.
  https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names
- **.NET Fiddle — Tarayıcıda C# çalıştırma**
  Bu haftanın sınıf örneklerini kurulum yapmadan hızlıca deneyebileceğiniz ortam.
  https://dotnetfiddle.net/